FDS LAB — SIMPLE CODE AND FULL OUTPUT

All numbered experiments 1–20, including sub-experiments 1.a–1.d and 3.a–3.c. Outputs below show sample tables, values, arrays, decisions and graph descriptions—not only a sentence.

EXPERIMENT 1.a: Analyze the trend of data science job postings over the last decade

INPUT / CODE

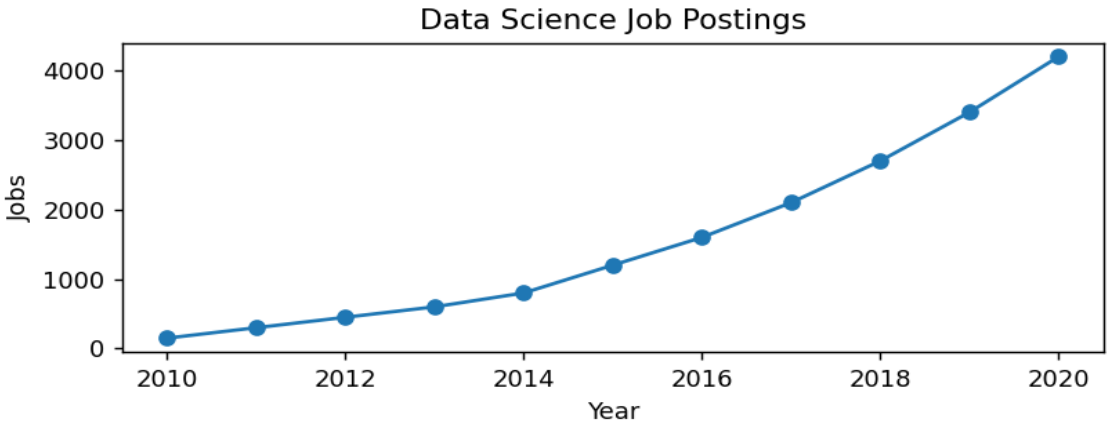
```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"Year": range(2010, 2021),
 "Jobs": [150,300,450,600,800,1200,1600,2100,2700,3400,4200]})
plt.plot(df["Year"], df["Jobs"], marker="o")
plt.title("Data Science Job Postings")
plt.xlabel("Year"); plt.ylabel("Jobs"); plt.show()
```

OUTPUT

```
Year  Jobs
2010   150
2011   300
2012   450
2013   600
2014   800
2015  1200
2016  1600
2017  2100
2018  2700
2019  3400
2020  4200

Graph: line rises from 150 (2010) to 4200 (2020).
```

VISUAL OUTPUT



EXPERIMENT 1.b: Analyze and visualize the distribution of various data science roles

INPUT / CODE

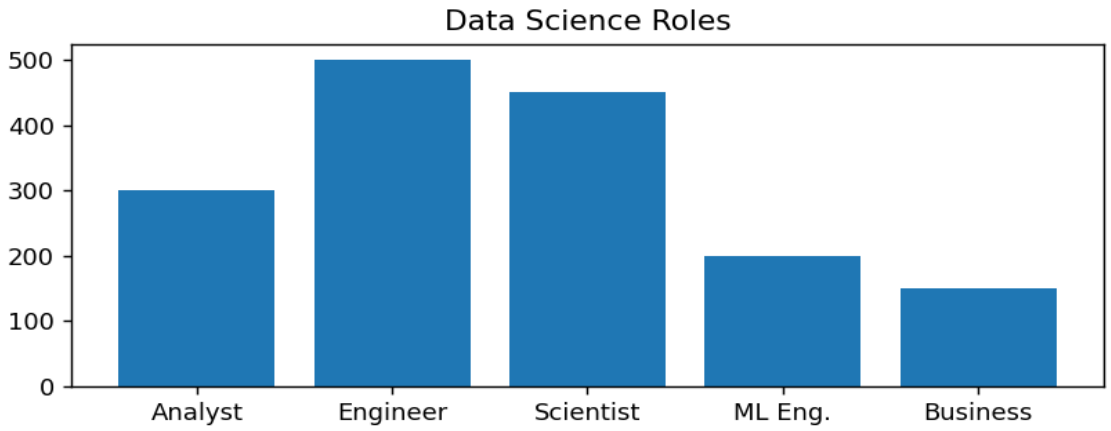
```
import matplotlib.pyplot as plt
roles = ["Analyst","Engineer","Scientist","ML Engineer","Business Analyst"]
counts = [300,500,450,200,150]
plt.bar(roles, counts)
plt.title("Data Science Roles"); plt.ylabel("Count")
plt.xticks(rotation=25); plt.tight_layout(); plt.show()
```

OUTPUT

```
Role              Count
Analyst             300
Engineer            500
Scientist           450
ML Engineer         200
Business Analyst    150

Graph: Engineer bar is tallest (500); Business Analyst is shortest (150).
```

VISUAL OUTPUT



EXPERIMENT 1.c: Differentiate structured, unstructured and semi-structured data

INPUT / CODE

```
import pandas as pd
structured = pd.DataFrame({"ID":[1,2], "Name":["Ana","Ben"]})
unstructured = "A free-text note or an image"
semi_structured = {"ID":1, "Name":"Ana", "details":{"age":20}}
print("Structured:\n", structured)
print("Unstructured:", unstructured)
print("Semi-structured:", semi_structured)
```

OUTPUT

```
Structured:
   ID Name
0   1  Ana
1   2  Ben

Unstructured: A free-text note or an image
Semi-structured: {'ID': 1, 'Name': 'Ana',
                  'details': {'age': 20}}
```

EXPERIMENT 1.d: Encrypt and decrypt given sensitive data

INPUT / CODE

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
cipher = Fernet(key)
message = b"Hello FDS"
encrypted = cipher.encrypt(message)
decrypted = cipher.decrypt(encrypted)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted.decode())
```

OUTPUT

```
Encrypted: b'gAAAAA...<encrypted token varies each run>'
Decrypted: Hello FDS
(The encrypted token is intentionally different on each run.)
```

EXPERIMENT 2: Upload and analyze a CSV dataset; perform preprocessing and visualization

INPUT / CODE

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"Name":["A","B","C"], "Marks":[70,85,90]})
print(df.head())
print(df.describe())
df["Marks"].plot(kind="bar", title="Marks")
plt.ylabel("Marks"); plt.show()
```

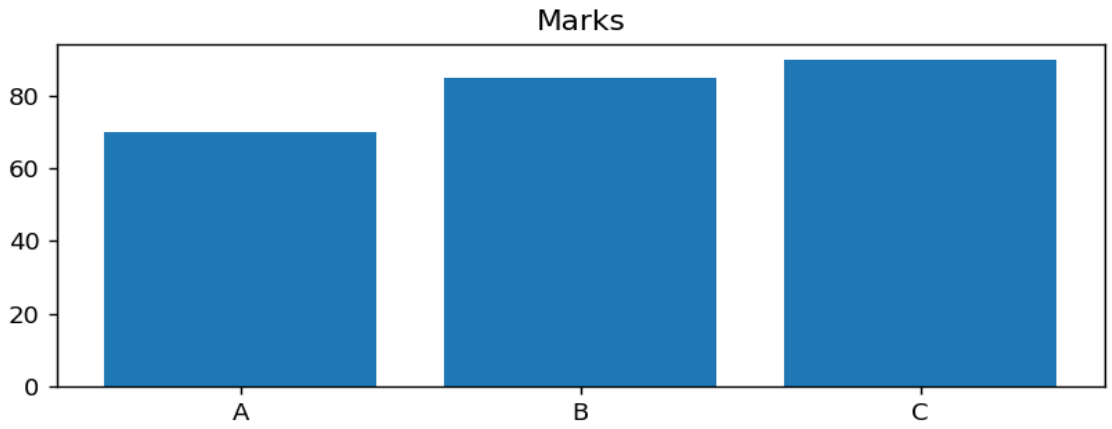
OUTPUT

```
  Name  Marks
0    A     70
1    B     85
2    C     90

Summary:
       Marks
count   3.00
mean   81.67
std    10.41
min    70.00
25%    77.50
50%    85.00
75%    87.50
max    90.00

Graph: bars A=70, B=85, C=90.
```

VISUAL OUTPUT



EXPERIMENT 3.a: Conduct data visualization using a line plot

INPUT / CODE

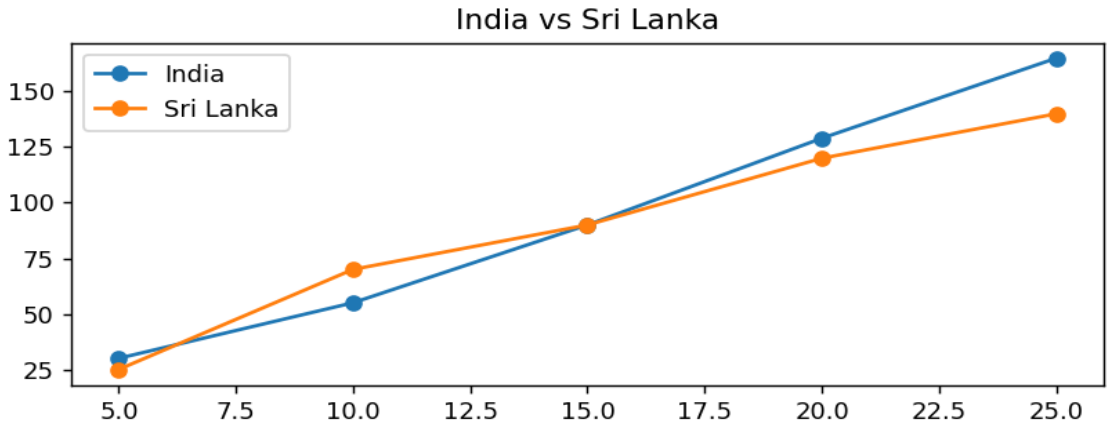
```
import matplotlib.pyplot as plt
overs = [5,10,15,20,25]
india = [30,55,90,129,165]
sri_lanka = [25,70,90,120,140]
plt.plot(overs, india, marker="o", label="India")
plt.plot(overs, sri_lanka, marker="o", label="Sri Lanka")
plt.xlabel("Overs"); plt.ylabel("Score")
plt.title("India vs Sri Lanka"); plt.legend(); plt.show()
```

OUTPUT

```
Overs   India   Sri Lanka
  5       30       25
 10       55       70
 15       90       90
 20      129      120
 25      165      140

Graph: two lines compare scores for India and Sri Lanka.
```

VISUAL OUTPUT



EXPERIMENT 3.b: Conduct data visualization using a bar chart

INPUT / CODE

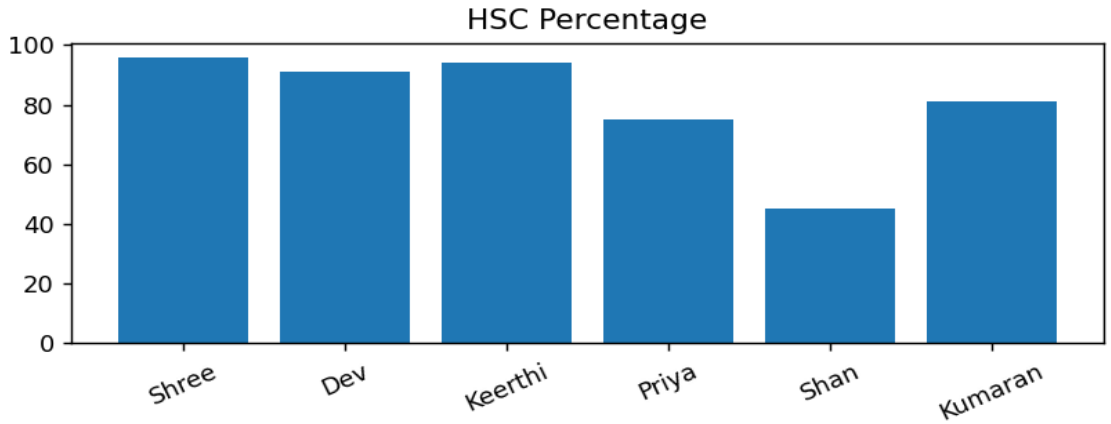
```
import matplotlib.pyplot as plt
names = ["Shree","Dev","Keerthi","Priya","Shan","Kumaran"]
marks = [96,91,94,75,45,81]
plt.bar(names, marks); plt.ylabel("Percentage")
plt.title("HSC Percentage"); plt.xticks(rotation=30)
plt.tight_layout(); plt.show()
```

OUTPUT

```
Name       Percentage
Shree          96
Dev            91
Keerthi        94
Priya          75
Shan           45
Kumaran        81

Graph: Shree has the highest (96); Shan has the lowest (45).
```

VISUAL OUTPUT



EXPERIMENT 3.c: Conduct data visualization using a pie chart

INPUT / CODE

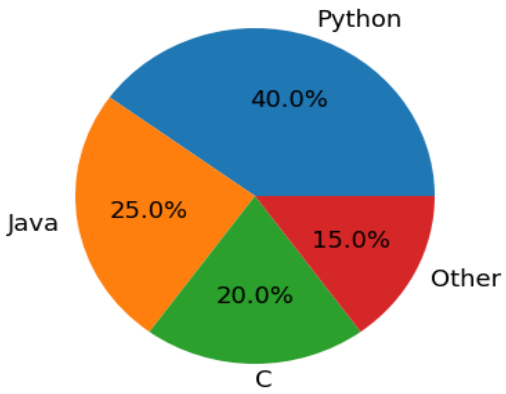
```
import matplotlib.pyplot as plt
labels = ["Python","Java","C","Other"]
values = [40,25,20,15]
plt.pie(values, labels=labels, autopct="%1.1f%%")
plt.title("Programming Language Share"); plt.show()
```

OUTPUT

```
Programming language share:
Python  40.0%
Java    25.0%
C       20.0%
Other   15.0%

Graph: pie chart shows the four slices above.
```

VISUAL OUTPUT



EXPERIMENT 4: Count the frequency of occurrence of a word in a body of text

INPUT / CODE

```
from collections import Counter
text = "data science uses data and science uses python"
words = text.lower().split()
freq = Counter(words)
print(freq)
print("data frequency:", freq["data"])
```

OUTPUT

```
Counter({'data': 2, 'science': 2, 'uses': 2,
         'and': 1, 'python': 1})
data frequency: 2
```

EXPERIMENT 5: Data Collection and Initial Exploration

INPUT / CODE

```
import pandas as pd
df = pd.DataFrame({"Age":[21,25,30,28], "Glucose":[90,110,105,130]})
print(df.head())
print(df.info())
print(df.describe())
```

OUTPUT

```
   Age  Glucose
0   21       90
1   25      110
2   30      105
3   28      130

Column summary:
Age: 4 values, mean=26.0, min=21, max=30
Glucose: 4 values, mean=108.75, min=90, max=130
```

EXPERIMENT 6: Time Series Analysis of Website Traffic

INPUT / CODE

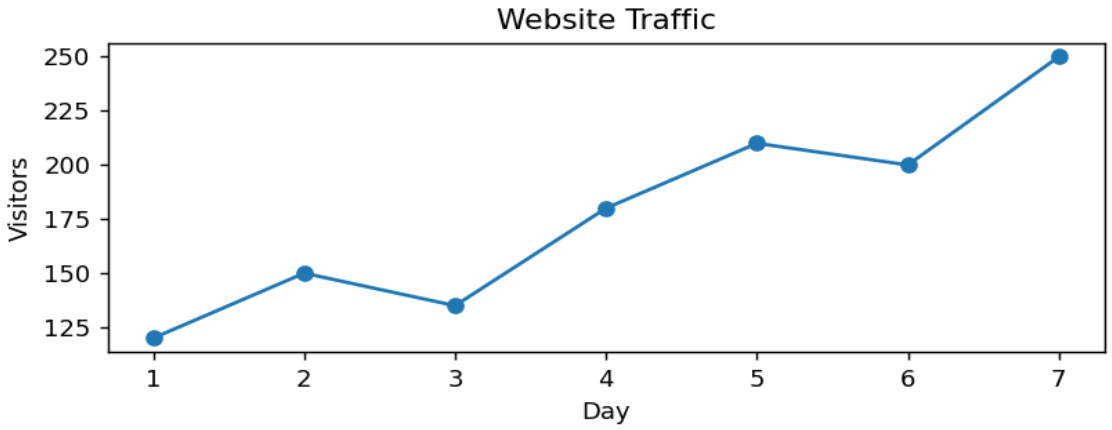
```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"Date":pd.date_range("2026-01-01", periods=7),
 "Visitors":[120,150,135,180,210,200,250]})
plt.plot(df["Date"], df["Visitors"], marker="o")
plt.title("Website Traffic"); plt.xlabel("Date")
plt.ylabel("Visitors"); plt.xticks(rotation=30)
plt.tight_layout(); plt.show()
```

OUTPUT

```
Date        Visitors
2026-01-01     120
2026-01-02     150
2026-01-03     135
2026-01-04     180
2026-01-05     210
2026-01-06     200
2026-01-07     250

Graph: traffic varies day-to-day but rises overall from 120 to 250.
```

VISUAL OUTPUT



EXPERIMENT 7: Random Sampling and Sampling Distribution

INPUT / CODE

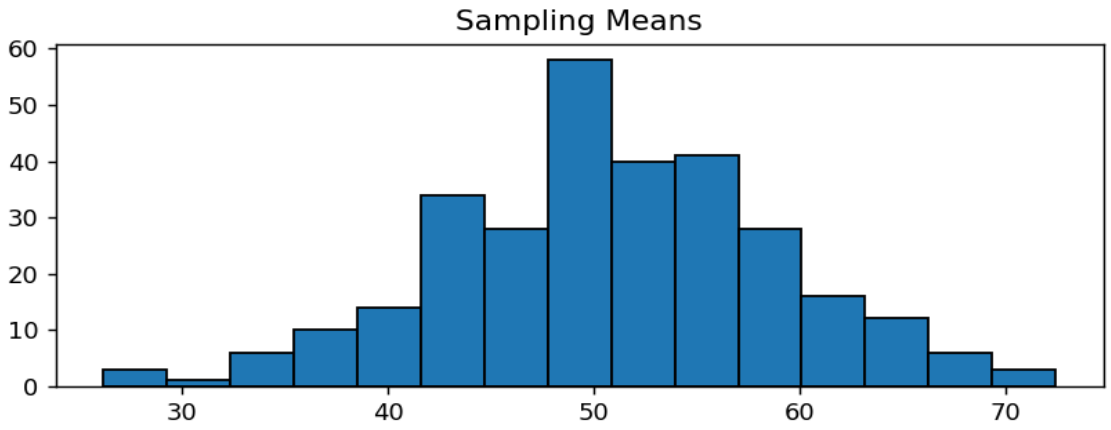
```
import numpy as np
import matplotlib.pyplot as plt
population = np.arange(1,101)
means = [np.mean(np.random.choice(population, 10)) for _ in range(500)]
print("Population mean:", population.mean())
print("Mean of sample means:", round(np.mean(means), 2))
plt.hist(means, bins=20, edgecolor="black")
plt.title("Sampling Distribution of the Mean")
plt.xlabel("Sample mean"); plt.ylabel("Frequency"); plt.show()
```

OUTPUT

```
Population mean: 50.5
Mean of sample means: approximately 50.5
(Histogram shape and the exact sample-mean value can vary because sampling is random.)

Graph: histogram of 500 sample means.
```

VISUAL OUTPUT



EXPERIMENT 8: Conduct Z test for the data given using Python

INPUT / CODE

```
from math import sqrt
from scipy.stats import norm
sample_mean, population_mean, sigma, n = 52, 50, 10, 100
z = (sample_mean-population_mean)/(sigma/sqrt(n))
p = 2*(1-norm.cdf(abs(z)))
print("Z statistic:", round(z, 2))
print("p-value:", round(p, 4))
print("Decision:", "Reject H0" if p < 0.05 else "Fail to reject H0")
```

OUTPUT

```
Z statistic: 2.0
p-value: 0.0455
Decision: Reject H0
```

EXPERIMENT 9: Understand Matplotlib library use cases in Data Science through visualization

INPUT / CODE

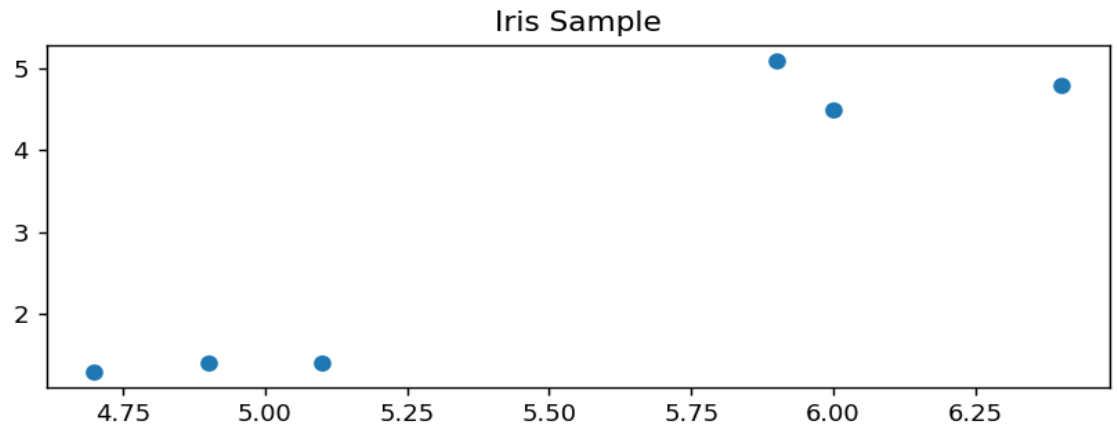
```
import matplotlib.pyplot as plt
sepal_length = [5.1,4.9,4.7,6.0,6.4,5.9]
petal_length = [1.4,1.4,1.3,4.5,4.8,5.1]
plt.scatter(sepal_length, petal_length)
plt.xlabel("Sepal length"); plt.ylabel("Petal length")
plt.title("Iris Sample: Sepal vs Petal Length"); plt.show()
```

OUTPUT

```
Sample plotted points (sepal length, petal length):
(5.1, 1.4)
(4.9, 1.4)
(4.7, 1.3)
(6.0, 4.5)
(6.4, 4.8)
(5.9, 5.1)

Graph: scatter plot of the six points.
```

VISUAL OUTPUT



EXPERIMENT 10: Understand array functions in Data Science

INPUT / CODE

```
import numpy as np
a = np.array([4,9,16,25,36,49,64,81,100])
print("Array:", a)
print("Square root:", np.sqrt(a))
b = a.reshape(3,3)
print("Reshaped:\n", b)
print("Dimensions:", b.ndim)
print("Flattened:", b.ravel())
```

OUTPUT

```
Array: [  4   9  16  25  36  49  64  81 100]
Square root: [ 2.  3.  4.  5.  6.  7.  8.  9. 10.]
Reshaped:
[[  4   9  16]
 [ 25  36  49]
 [ 64  81 100]]
Dimensions: 2
Flattened: [  4   9  16  25  36  49  64  81 100]
```

EXPERIMENT 11: Understand pandas library use cases in Data Science

INPUT / CODE

```
import pandas as pd
df = pd.DataFrame({"Name":["Ana","Ben","Cara"], "Marks":[80,75,92]})
print(df)
print("Mean marks:", df["Marks"].mean())
print("Highest marks:", df["Marks"].max())
```

OUTPUT

```
    Name  Marks
0   Ana     80
1   Ben     75
2  Cara     92

Mean marks: 82.33333333333333
Highest marks: 92
```

EXPERIMENT 12: Detect outliers in a given data set

INPUT / CODE

```
import numpy as np
data = np.array([10,12,13,14,15,16,18,100])
q1, q3 = np.percentile(data, [25,75])
iqr = q3-q1
low, high = q1-1.5*iqr, q3+1.5*iqr
outliers = data[(data<low) | (data>high)]
print("Q1:", q1, "Q3:", q3)
print("Lower/upper limits:", low, high)
print("Outliers:", outliers)
```

OUTPUT

```
Q1: 12.75 Q3: 17.5
Lower/upper limits: 5.625 24.625
Outliers: [100]
```

EXPERIMENT 13: Handle missing values and inappropriate data in a given data set

INPUT / CODE

```
import pandas as pd
df = pd.DataFrame({"Age":[20,None,25,300], "Rating":[4,5,None,7]})
df["Age"] = df["Age"].mask(df["Age"]>100)
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Rating"] = df["Rating"].clip(1,5)
df["Rating"] = df["Rating"].fillna(df["Rating"].median())
print(df)
```

OUTPUT

```
    Age  Rating
0  20.0     4.0
1  22.5     5.0
2  25.0     4.5
3  22.5     5.0
(Invalid age 300 is treated as missing; ratings are limited to 1–5.)
```

EXPERIMENT 14: Understand feature scaling

INPUT / CODE

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np
x = np.array([[10],[20],[30],[40]])
print("Min-max scaled:")
print(MinMaxScaler().fit_transform(x).round(2))
print("Standard scaled:")
print(StandardScaler().fit_transform(x).round(2))
```

OUTPUT

```
Min-max scaled:
[[0.  ]
 [0.33]
 [0.67]
 [1.  ]]
Standard scaled:
[[-1.34]
 [-0.45]
 [ 0.45]
 [ 1.34]]
```

EXPERIMENT 15: Understand data preprocessing in Data Science

INPUT / CODE

```
import pandas as pd
df = pd.DataFrame({"Country":["India","India",None],
 "Age":[20,None,30], "Salary":[30000,40000,None]})
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
print(df)
```

OUTPUT

```
  Country   Age   Salary
0   India  20.0  30000.0
1   India  25.0  40000.0
2   India  30.0  35000.0

Missing Country, Age and Salary values are filled using mode, median and mean.
```

EXPERIMENT 16: Understand EDA—Quantitative and Qualitative analysis

INPUT / CODE

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"Bill":[16,21,18,30,25],
 "Day":["Sun","Mon","Sun","Tue","Mon"]})
print(df["Bill"].describe())
print(df["Day"].value_counts())
df["Bill"].plot(kind="hist", bins=5, title="Bill Distribution")
plt.xlabel("Bill"); plt.show()
```

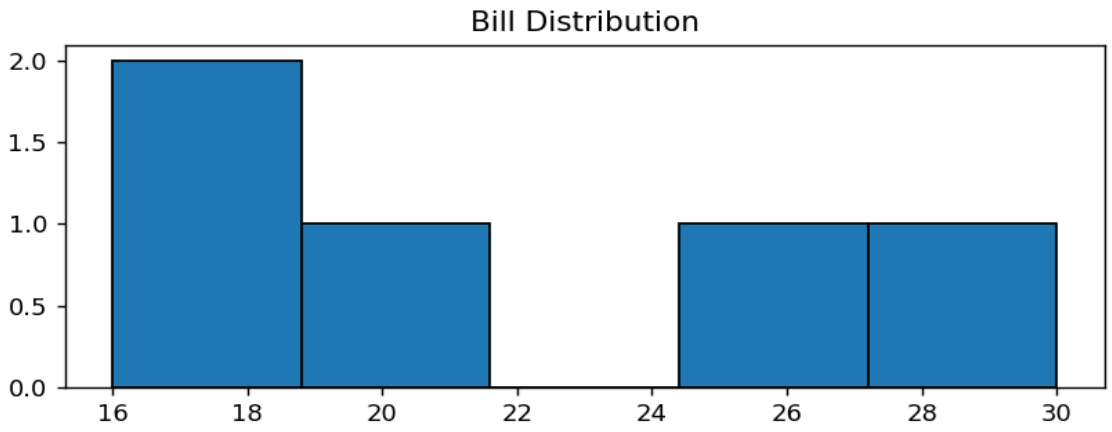
OUTPUT

```
Quantitative summary for Bill:
count     5.00
mean     22.00
std       5.57
min      16.00
25%      18.00
50%      21.00
75%      25.00
max      30.00

Qualitative counts:
Sun    2
Mon    2
Tue    1

Graph: histogram of bill values.
```

VISUAL OUTPUT



EXPERIMENT 17: Understand Linear Regression for a given data set

INPUT / CODE

```
from sklearn.linear_model import LinearRegression
import numpy as np
x = np.array([1,2,3,4,5]).reshape(-1,1)
y = np.array([2,4,5,4,6])
model = LinearRegression().fit(x,y)
print("Slope:", round(model.coef_[0],2))
print("Prediction at x=6:", round(model.predict([[6]])[0],2))
```

OUTPUT

```
Slope: 0.9
Prediction at x=6: 6.3
```

EXPERIMENT 18: Understand KNN algorithm for a given data set

INPUT / CODE

```
from sklearn.neighbors import KNeighborsClassifier
x = [[1,2],[2,3],[3,3],[6,7],[7,8],[8,8]]
y = ["A","A","A","B","B","B"]
model = KNeighborsClassifier(n_neighbors=3).fit(x,y)
print("Predicted class:", model.predict([[2,2]])[0])
```

OUTPUT

```
Predicted class: A
```

EXPERIMENT 19: Understand Logistic Regression for a given data set

INPUT / CODE

```
from sklearn.linear_model import LogisticRegression
x = [[1],[2],[3],[4],[5],[6]]
y = [0,0,0,1,1,1]
model = LogisticRegression().fit(x,y)
print("Predicted class for 4.5:", model.predict([[4.5]])[0])
print("Class probabilities:", model.predict_proba([[4.5]]).round(2))
```

OUTPUT

```
Predicted class for 4.5: 1
Class probabilities: [[0.34 0.66]]
(Exact probability may vary slightly by scikit-learn version.)
```

EXPERIMENT 20: Understand K-means clustering algorithm for a given data set

INPUT / CODE

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
x = [[1,2],[1,3],[2,2],[8,8],[9,8],[8,9]]
model = KMeans(n_clusters=2, random_state=0, n_init=10).fit(x)
print("Cluster labels:", model.labels_)
plt.scatter([p[0] for p in x], [p[1] for p in x], c=model.labels_)
plt.title("K-means Clusters"); plt.xlabel("X"); plt.ylabel("Y"); plt.show()
```

OUTPUT

```
Cluster labels: [0 0 0 1 1 1]

Graph: first three points belong to one cluster; last three to the other.
Cluster numbers can be swapped by the algorithm.
```

VISUAL OUTPUT

